



Machine Learning Session Summary 10-07-2023

- In this session, we will use mediapipe library for object detection and recognition task.
- **Installation:**
 - *pip install mediapipe*
 - *pip install cvzone*
- We will use cvzone to detect hand using `HandTrackingModule.HandDetector`

```
In [ ]: 1 HandDetector()

Init signature: HandDetector(mode=False, maxHands=1, detectionCon=0.5, minTrackC
on=0.5)
Docstring:
Finds Hands using the mediapipe library. Exports the landmarks
in pixel format. Adds extra functionalities like finding how
many fingers are up or the distance between two fingers. Also
provides bounding box info of the hand found.
Init docstring:
:param mode: In static mode, detection is done on each image: slower
:param maxHands: Maximum number of hands to detect

In [3]: 1 from cvzone.HandTrackingModule import HandDetector

In [4]: 1 handModel = HandDetector(maxHands=1)
```

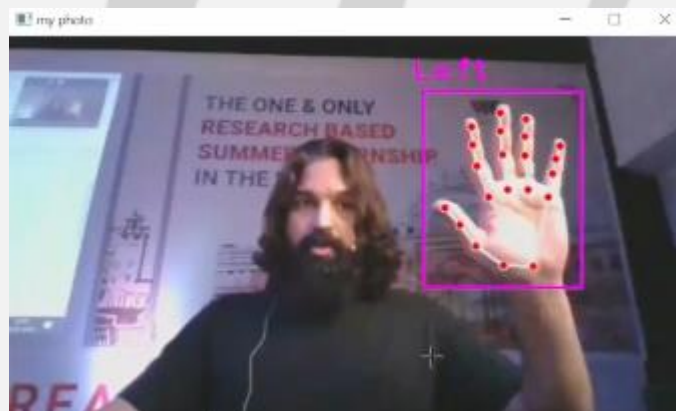
- HandDetector() class will help to create a module that will detect hand as well as tell which hand it is and the coordinates.

```
In [8]: 1 cv2.imshow("my photo" , photo)
        2 cv2.waitKey()
        3 cv2.destroyAllWindows()

In [10]: 1 handModel.findHands(photo)

Out[10]: ([{'lmList': [[530, 186, 0],
                      [499, 184, -20],
                      [471, 166, -29],
                      [455, 144, -36],
                      [437, 127, -41],
                      [480, 113, -14],
                      [468, 80, -24],
                      [464, 57, -33],
                      [463, 37, -40],
                      [499, 186, -11]]}]])
```

- handModel.findHands() – will give you landmark of hand, other is draw box on the hand locating all points if hand is there otherwise same photo is returned.



- Hand Detection code:

```
In [32]: 1 if hand:
        2     print("hand detected", hand[0]['type'])
        3     cv2.imshow("my photo" , image)
        4     cv2.waitKey()
        5     cv2.destroyAllWindows()
        6 else:
        7     print("no hand detected")

hand detected
```

```
In [40]: 1 hand[0][1]

Out[40]: {'lmList': [[93, 141, 0],
                    [129, 142, -11],
                    [161, 124, -11],
                    [181, 104, -9],
                    [196, 93, -7],
                    [152, 83, 1],
                    [174, 61, 0],
                    [186, 47, -2],
                    [194, 35, -4],
                    [134, 72, 3],
```



- This model can also help us to detect fingersUp in the image captured that have hand.

```
In [69]: 1 if hand:
          2     print("detected hand : " , hand[0]['type'])
          3

detected hand : Left

In [72]: 1 handModel.fingersUp(hand[0])

Out[72]: [1, 1, 1, 1, 1]
```

- Fingers Up detection code

```

In [ ]: 1 while True:
        2     status , photo = cap.read()
        3     hand = handModel.findHands(photo, draw=False )
        4     I
        5     if hand:
        6         totalFinger = handModel.fingersUp(hand[0])
        7
        8         if totalFinger == [ 1 , 0 , 0 , 0 , 0 ]:
        9             print("thumbs up")
        10
        11         elif totalFinger == [ 0 , 1 , 0 , 0 , 0 ]:
        12             print("index finger")
        13             os.system("chrome")
        14

```

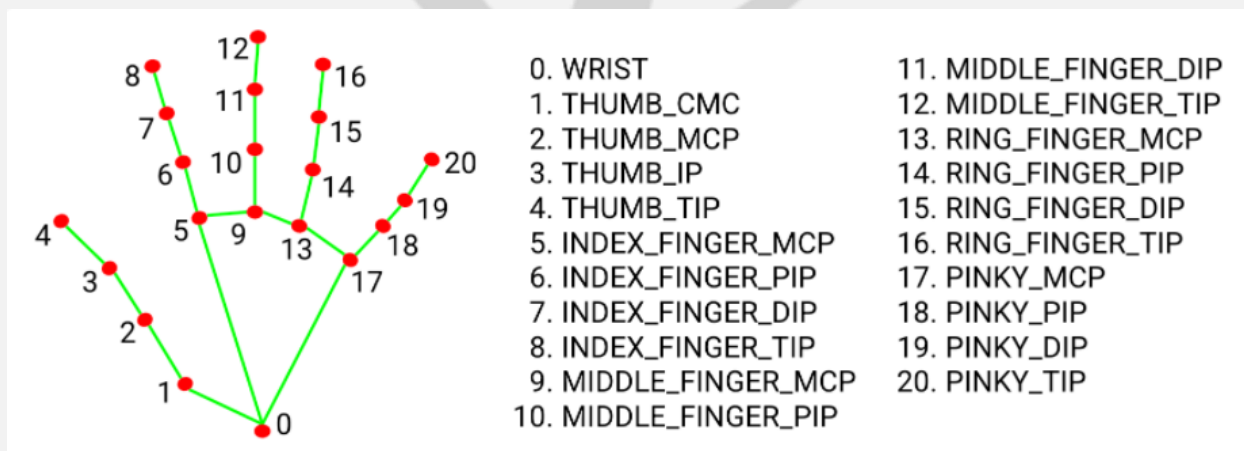
➤ Landmark list returned by HandDetection()

```

In [40]: 1 hand[0][
Out[40]: {'lmList': [[93, 141, 0],
                    [129, 142, -11],
                    [161, 124, -11],
                    [181, 104, -9],
                    [196, 93, -7],
                    [152, 83, 1],
                    [174, 61, 0],
                    [186, 47, -2],
                    [194, 35, -4],
                    [134, 72, 3],

```

➤ The rows tell us about the position of each predefined points of hands.



- That is the list of 21 points.
- Other models we can implement is Face Mesh, Pose Detection, and others.